Laboratory Report 5: Dropout, Batch Normalization, and Optimizers

Course: COMP-341L - Artificial Neural Networks Lab
Lab Assignment: 5
Topic: Implementing Dropout, Batch Normalization, and Optimizers in Deep Networks
Submission Date: February 16, 2026

Student Name: Zarmeena Jawad
Roll Number: B23F0115AI125
Section: AI Red

Academic integrity: I declare that this report is my own work. I have not copied from other students or submitted another person's report. As per course policy, plagiarism or copy-pasting leads to zero marks for everyone involved.

Executive Summary

This laboratory focuses on improving the training and generalization of deep multilayer perceptrons (MLPs) using TensorFlow. We use the Breast Cancer classification dataset and implement Dropout (regularization), Batch Normalization (training stability), and a comparison of optimizers (SGD vs Adam). We also study the effect of different learning rates on convergence and stability. All experiments are run for 50 epochs; training and validation loss and accuracy are plotted for each configuration.

Learning Objectives

- Build and train deep neural networks in TensorFlow on a real dataset (e.g. Breast Cancer).
- Implement Dropout and explain how it reduces overfitting.
- Implement Batch Normalization and relate it to convergence speed and stability.

- Compare SGD and Adam optimizers in terms of speed, smoothness, and final performance.
- Use validation curves to diagnose overfitting.
- Observe how learning rate choice affects stability and possible divergence.

Experimental Setup

- **Dataset:** Breast Cancer (sklearn). Binary classification (malignant vs benign).
- **Preprocessing:** 80% train / 20% validation (stratified). Features standardized with `StandardScaler`.
- **Base architecture:** Input (30) → Dense(64, ReLU) → Dense(64, ReLU) → Dense(32, ReLU) → Dense(1, sigmoid).
- **Training:** 50 epochs, batch size 32, binary cross-entropy. Default: Adam, lr=0.001.

Code: `lab5_experiments.py`. Run from repo root: `./venv/bin/python3 "Lab 5/Zarmeena Jawad's Lab/lab5_experiments.py"` or from this folder: `../../venv/bin/python3 lab5_experiments.py`

Task 1: Baseline Model (No Dropout, No BatchNorm)

We build a deep network with no Dropout and no BatchNorm, train for 50 epochs, and plot training loss, validation loss, and accuracy.

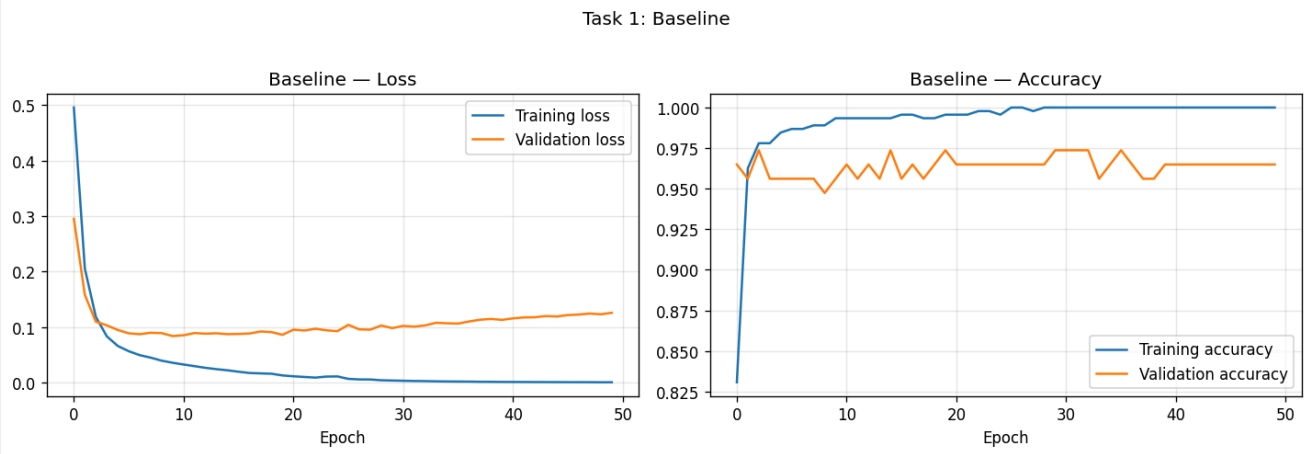
Implementation

The baseline is a sequential model with three hidden layers (64, 64, 32 units) and ReLU, then a sigmoid output. The script uses `baseline_mlp(input_dim)` and `run_training()`.

Findings

The baseline often shows a growing gap between training and validation loss (overfitting).

Figure 1.1: Task 1 – Training loss, validation loss, and accuracy.



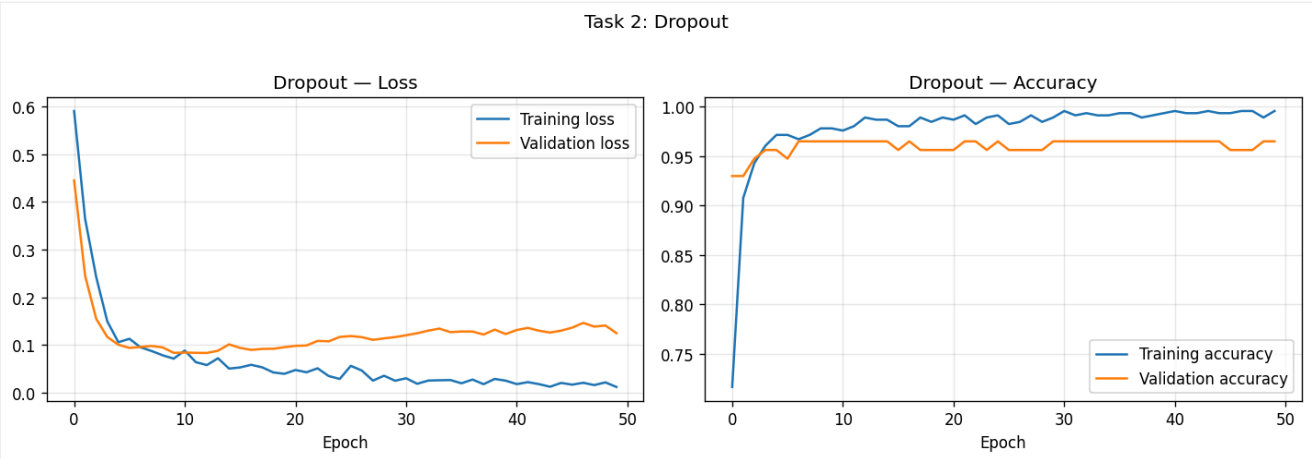
Task 2: Add Dropout

We add Dropout (rate 0.3) after each hidden layer and compare overfitting behavior and accuracy with the baseline.

Comparison (overfitting behavior and accuracy)

- **Overfitting behavior:** Validation loss tracks training loss better; train–validation gap is smaller.
- **Accuracy:** Validation accuracy comparable or better (see Comparison Table).

Figure 2.1: Task 2 – Dropout.



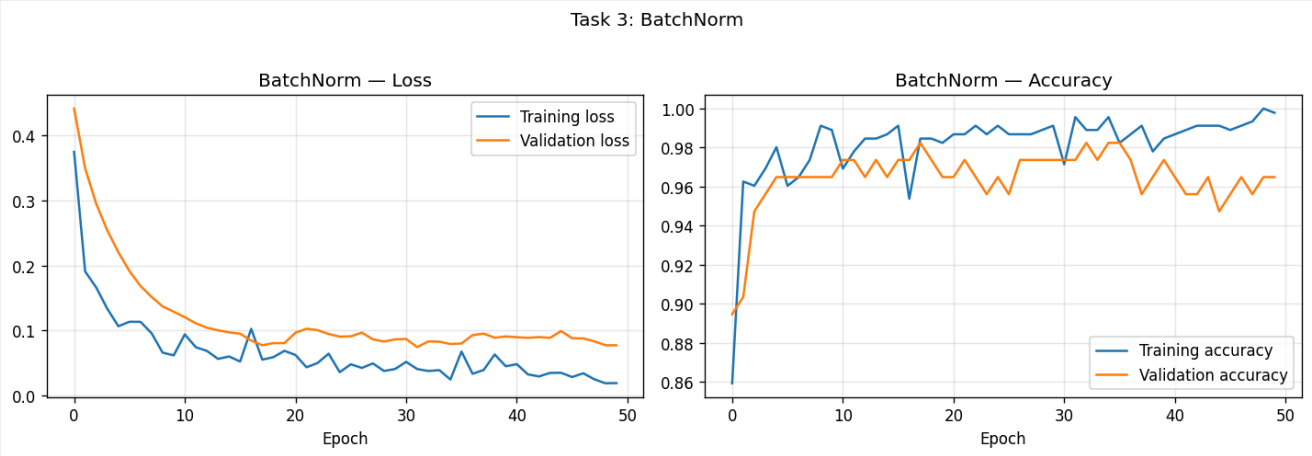
Task 3: Add BatchNorm

We add BatchNorm after each Dense layer (before ReLU) and compare convergence speed, stability, and final performance.

Comparison (convergence speed, stability, final performance)

- **Convergence speed:** Loss decreases faster and more steadily.
- **Stability:** Curves are smoother and less noisy.
- **Final performance:** Validation loss and accuracy improve (see Comparison Table).

Figure 3.1: Task 3 – BatchNorm.



Task 4: Combine Dropout and BatchNorm

We use both BatchNorm and Dropout in the same model, train, and analyze.

Figure 4.1: Task 4 – Dropout + BatchNorm.

Task 5: Optimizer Comparison (SGD vs Adam)

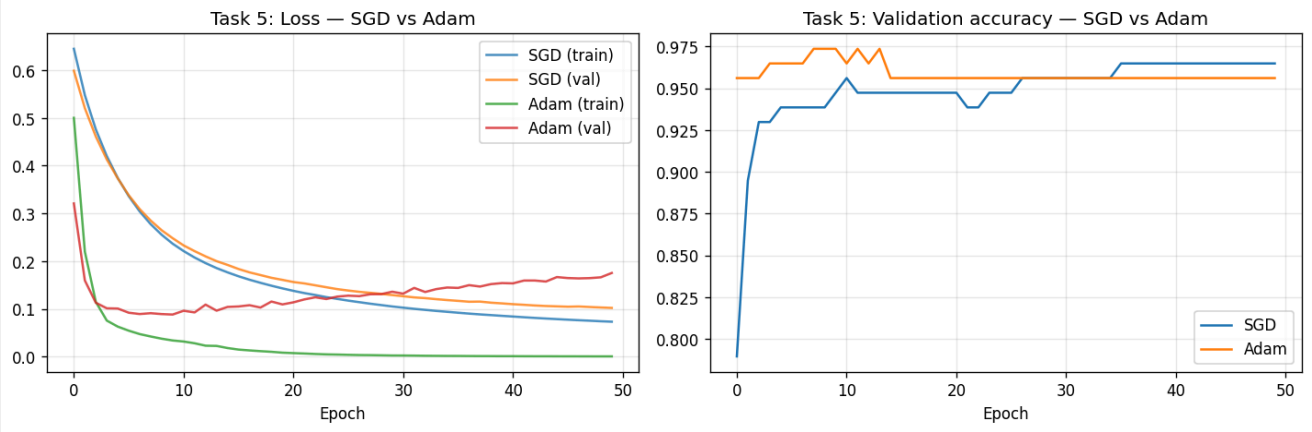
We train the same baseline with SGD (lr=0.01) and Adam (lr=0.001), plot loss curves on the same graph, and compare speed, smoothness, and final performance.

Comparison

- **Speed:** Adam usually reaches a good validation loss in fewer epochs.

- **Smoothness:** Adam's curves are typically smoother; SGD can show more oscillation.
- **Final performance:** See Comparison Table.

Figure 5.1: Task 5 – Loss curves on same graph (SGD vs Adam).

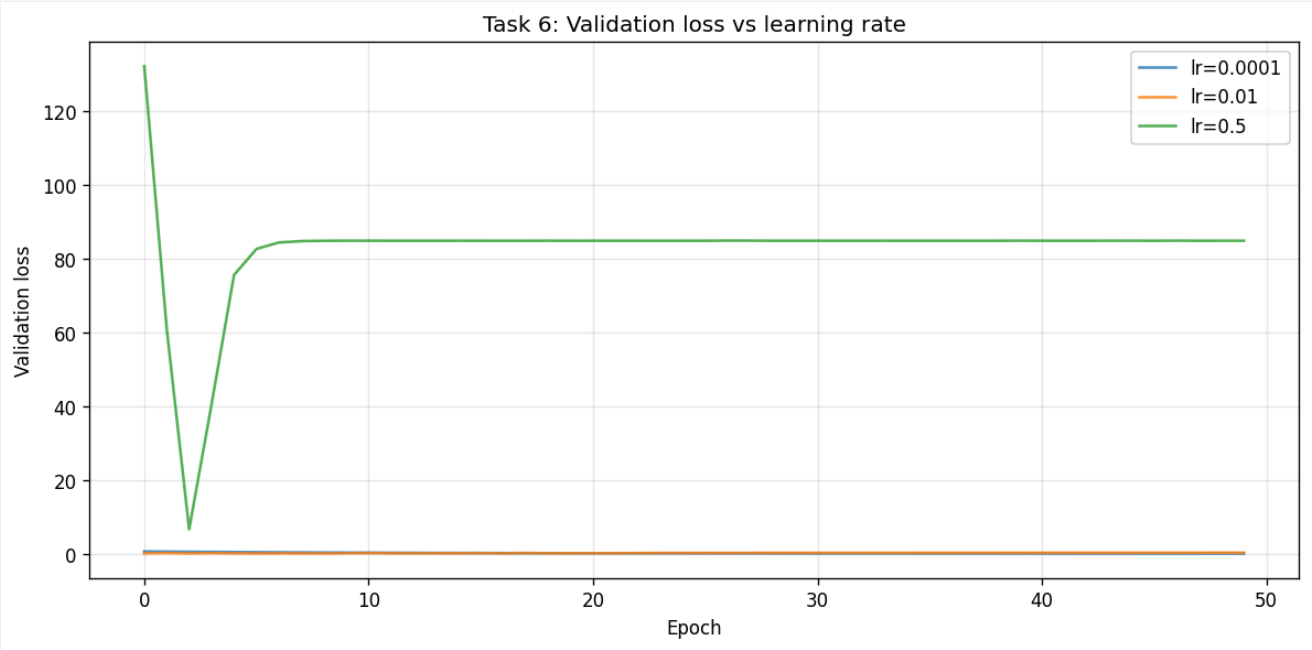


Task 6: Learning Rate Sensitivity

We train the baseline with Adam using lr = 0.0001, 0.01, and 0.5 and observe stability or divergence.

- **lr = 0.0001:** Stable but slow.
- **lr = 0.01:** Good convergence, generally stable.
- **lr = 0.5:** Can diverge or oscillate; illustrates instability.

Figure 6.1: Task 6 – Learning rate sensitivity.



Comparison Table (Final Epoch)

Model	Val Loss	Val Acc
Baseline	0.1260	0.9649
Dropout	0.1247	0.9649
BatchNorm	0.0773	0.9649
Dropout+BatchNorm	0.0951	0.9561
SGD (lr=0.01)	0.1021	0.9649
Adam (lr=0.001)	0.1753	0.9561

All models trained for 50 epochs.

Results (All Loss Plots)

All plots are embedded above. Reference: task1_baseline.png, task2_dropout.png, task3_batchnorm.png, task4_combined.png, task5_optimizer_comparison.png, task6_learning_rate_sensitivity.png in the `plots/` folder.

Discussion

Overfitting and regularization: The baseline overfits; Dropout reduces the gap by randomly disabling neurons. BatchNorm stabilizes layer inputs and speeds convergence; combining both gives stable training and good validation performance.

Optimizers: Adam converges quickly and smoothly; SGD can reach similar accuracy with more epochs and more oscillation.

Learning rate: Too small slows convergence; too large can cause divergence. A moderate value (e.g. 0.001 for Adam) works well.

Conclusions

We implemented and compared Dropout, BatchNorm, and optimizers on the Breast Cancer dataset. The baseline overfit; Dropout improved generalization; BatchNorm improved convergence and stability; the combined model gave a good balance. Adam converged faster and more smoothly than SGD. Learning rate experiments showed that an appropriate value is essential for stable training.

References

- Lab 05 Manual (Lab 05.pdf): Dropout, Batch Normalization, Optimizers, Lab Tasks (Tasks 1–6).